



# Unity GC 系列教程第一篇：GC 基础概念与工作原理

原创 已于 2025-07-25 10:52:10 修改 · 1.3k 阅读 ·  28 ·  22 · CC 4.0 BY-SA版权

文章标签： [#unity](#) [#游戏引擎](#) [#架构](#) [#c#](#) [#开发语言](#)

Unity GC 系列教程第一篇：GC 基础概念与工作原理

Unity GC 系列教程第二篇：Unity 中常见的 GC Alloc 场景与分析工具

Unity GC 系列教程第三篇：GC Alloc 优化技巧与实践（上）

Unity GC 系列教程第四篇：GC Alloc 优化技巧与实践（下）与 GC 调优

Unity GC 系列教程第五篇：高级 GC 内核

欢迎来到 [Unity](#) GC 系列教程的第一篇！作为一名 Unity 开发者，你可能已经或多或少遇到过游戏卡顿、[帧率](#) 骤降等问题。这些问题背后，**GC (Garbage Collection)** 常常是罪魁祸首之一。理解 GC，掌握其工作原理，是优化 Unity 游戏性能、提升用户体验的关键一步。

在本篇中，我们将深入探讨 GC 的基础概念、它为何存在、以及其核心的工作原理。我们将从宏观的角度理解 GC 的必要性，然后逐步剖析其内部机制，为后续的优化实践打下坚实的基础。

## 1.1 什么是 GC (Garbage Collection)? 我们为何需要它?

在编程世界中，内存管理是一个永恒的话题。当我们创建变量、对象、数据结构时，它们都需要占用内存空间。程序运行结束后，或者这些变量、对象不再被需要时，它们所占用的内存就需要被释放，以便其他程序或对象可以使用。

手动管理内存是一项复杂且容易出错的任务。在 C++ 等语言中，开发者需要自己负责内存的分配 ([new](#)) 和释放 ([delete](#))。如果忘记释放内存，就会导致 **内存泄漏 (Memory Leak)**，即程序占用的内存越来越多，最终可能导致系统崩溃或程序运行缓慢。反之，如果过早地释放了还在使用的内存，则可能导致 **悬空指针 (Dangling Pointer)**，引发程序崩溃或不可预测的行为。

为了解决这些手动内存管理带来的痛点，**垃圾回收 (Garbage Collection)** 技术应运而生。GC 的核心思想是：**自动识别并回收程序中不再使用的内存**。开发者不再需要手动编写内存释放的代码，而是由 GC 运行时系统（在 C# 中是 .NET / Mono 运行时）自动完成这项工作。

想象一下你有一个房间，里面堆满了各种物品。如果你需要自己决定哪些物品要扔掉，哪些要保留，这会很累。但如果有一个智能机器人（GC）能自动识别并清理那些你不再使用的物品，你的房间就能保持整洁，你也能更专注于你的工作。GC 在程序内存管理中的作用就类似于这个智能机器人。

在 C# 和 Unity 中，我们主要处理的是 **托管内存 (Managed Memory)**。这意味着我们创建的大部分对象（如类实例、字符串、数组等引用类型）都由 .NET / Mono 运行时进行内存管理。而像直接操作底层硬件或文件句柄等少数情况，可能会涉及到 **非托管内存 (Unmanaged Memory)**，这部分内存通常需要我们手动管理或使用特定的 [API](#) 来释放。本系列教程主要关注托管内存的 GC 优化。

GC 的基本目标是：

- **自动化内存管理**：将开发者从繁琐的内存管理任务中解放出来。
- **防止内存泄漏**：自动回收不再使用的内存，避免程序占用过多资源。
- **提高程序稳定性**：减少因内存管理错误（如野指针、二次释放）导致的崩溃。

## 1.2 GC 的基本原理：可达性分析与常见算法

尽管 GC 的具体实现非常复杂，但其核心原理是基于 **可达性分析 (Reachability Analysis)**。简单来说，GC 会判断程序中的某个对象是否仍然“可达”，即从程序的“根”出发，是否还能找到并访问到这个对象。

### 1.2.1 GC 根 (GC Roots)

要理解可达性分析，首先要明白什么是 **GC 根 (GC Roots)**。GC 根是程序中那些无论如何都不会被 GC 回收的对象。它们是 GC 判断其他对象是否“存活”的起点。常见的 GC 根包括：

- **栈变量 (Stack Variables)**：当前正在执行的方法中的局部变量。
- **静态变量 (Static Variables)**：全局可访问的变量，它们在程序整个生命周期内都可能存在。
- **CPU 寄存器 (CPU Registers)**：CPU 内部存储数据的寄存器。
- **JNI 引用 (JNI References)**（在 [Java](#) 等语言中，但在 Unity 的跨语言交互中也可能遇到类似概念）。
- **其他特殊系统对象**：例如，由运行时或外部系统（如操作系统、[Unity 引擎](#) 本身）直接引用的对象。

你可以将 GC 根想象成一棵树的树根，所有从这些树根延伸出来的枝干（引用）所连接到的叶子（对象），都是“活着的”对象。

### 1.2.2 可达性分析算法

GC 在执行时，会从这些 GC 根开始，遍历所有它们直接或间接引用的对象。所有能够被遍历到的对象，都被认为是“可达的”，也就是“存活的”。而那些无法从任何 GC 根到达的对象，则被认为是“不可达的”，也就是“垃圾”，可以被回收。

这个过程就像你从一本书的目录（GC 根）开始，翻阅所有相关的章节和参考文献（引用），所有能找到的内容都是有用的。而那些目录中没有提及，也没有被其他章节引用的内容，就是多余的，可以扔掉。

### 1.2.3 常见的 GC 算法简介

虽然 Unity 使用的 Mono GC 基于这些算法的变种，但了解这些基础算法有助于理解 GC 的工作方式：

- **标记-清除 (Mark-Sweep)**

- **标记 (Mark) 阶段**：从 GC 根开始，遍历所有可达对象，并将其标记为“存活”。
- **清除 (Sweep) 阶段**：遍历整个堆内存，回收所有未被标记（即不可达）的对象所占用的内存空间。
- **优点**：实现相对简单。
- **缺点**：
  - **内存碎片 (Memory Fragmentation)**：回收后内存空间不连续，可能会产生大量小块的“碎片”，导致后续分配大对象时找不到足够的连续空间。
  - **GC 暂停 (GC Pause)**：标记和清除过程都需要暂停应用程序的执行（Stop-The-World），这在游戏等实时应用中是不可接受的。

- **复制 (Copying)**

- 将堆内存划分为两个区域：**From 空间 (From Space)** 和 **To 空间 (To Space)**。
- GC 执行时，将所有存活对象从 From 空间复制到 To 空间。
- 复制完成后，From 空间中的所有对象都被清除，From 空间和 To 空间的角色互换。
- **优点**：

- **没有内存碎片**：复制过程会将存活对象紧凑排列，解决了内存碎片问题。
- **回收效率高**：只需要关注存活对象，而不是整个堆。
- **缺点**：
  - **内存利用率低**：至少需要两倍的内存空间来存储数据。
  - **复制开销**：如果存活对象较多，复制的开销会很大。
- **标记-整理 (Mark-Compact)**
  - 结合了标记-清除和复制算法的优点。
  - **标记 (Mark) 阶段**：与标记-清除相同，标记所有存活对象。
  - **整理 (Compact) 阶段**：将所有存活对象向堆的一端移动，使它们紧密排列，然后直接清理掉端部之外的内存。
  - **优点**：
    - **没有内存碎片**。
    - **内存利用率高**：不需要额外的两倍空间。
  - **缺点**：
    - **GC 暂停时间较长**：标记和整理都需要暂停应用程序。移动对象涉及到更新所有引用，开销较大。

#### 1.2.4 分代 GC (Generational GC)

现代的 GC 算法（包括 .NET/Mono 的 GC）通常采用 **分代 GC** 的策略。这是基于两个重要的观察：

1. **大部分对象生命周期都很短**：很多临时对象（如方法内部的局部变量、临时计算结果）在创建后很快就不再被使用。
2. **存活时间越长的对象，未来存活的概率越大**：如果一个对象已经存活了很长时间，它很可能是一个重要且会被持续使用的对象。

分代 GC 将堆内存划分为不同的“代”：

- **新生代 (Young Generation / Gen 0)**：存放新创建的对象。这里通常采用 **复制算法**。因为新生代中对象死亡率很高，复制算法在这种情况下效率很高（只需复制少量存活对象）。GC 在新生代执行的频率最高，暂停时间最短。
- **老年代 (Old Generation / Gen 1 / Gen 2)**：存放经过多次新生代 GC 后仍然存活的对象。这些对象被认为是“稳定”的，其 GC 频率较低，但因为对象数量可能较多，每次 GC 的暂停时间可能较长，通常采用**标记-整理或标记-清除算法**。

分代 GC 的优势在于：

- **减少 GC 暂停时间**：对新生代的频繁、快速 GC 减少了大部分对象的回收时间。
- **提高 GC 效率**：针对不同生命周期特性的对象采用不同的回收策略。

### 1.3 Unity 中的 Mono GC

Unity 长期以来主要使用 **Mono 运行时**。Mono 是 .NET Framework 的**开源** 实现，因此 Unity 中的 GC 行为很大程度上继承了 Mono 的特性。

早期的 Unity 版本（直到 Unity 2019 引入 Incremental GC 之前），Unity 中的 Mono GC 主要是 **阻塞式 (Blocking) GC**。这意味着当 GC 需要执行时，它会暂停所有游戏逻辑的执行（即 **Stop-The-World**），直到垃圾回收完成。对于实时性要求高的游戏来说，这种突然的暂停会直接导致帧率骤降，产生明显的卡顿，严重影响用户体验。

想象一下你正在玩一个紧张刺激的动作游戏，突然屏幕卡住了半秒甚至一秒，那体验简直是灾难性的！这就是阻塞式 GC 可能带来的问题。

虽然在 Unity 2019 之后引入了 **Incremental GC**（增量式 GC），极大地缓解了这个问题，但理解阻塞式 GC 的概念仍然非常重要，因为它帮助我们理解为什么优化 GC Alloc 如此关键。即使有了增量式 GC，频繁的内存分配仍然会给 GC 带来压力。

### 1.4 为什么 GC 会产生性能问题？

现在我们已经对 GC 的原理有了一些了解，那么为什么它会成为 Unity 游戏开发中的一个主要性能瓶颈呢？主要有两个原因：

#### 1.4.1 GC 暂停 (GC Pause / Stop-The-World)

正如前面提到的，无论是标记、清除、复制还是整理，GC 的某些阶段都需要暂停应用程序的执行。这个暂停就是 **GC 暂停 (GC Pause)**。

- **在阻塞式 GC 中**：GC 会一次性完成所有工作，导致较长时间的暂停。暂停时间取决于堆内存的大小、存活对象的数量以及GC的复杂性。在堆内存达到某个阈值时，GC 会被

触发。

- **在增量式 GC 中：**GC 将工作分解为多个小块，在多帧中逐步执行，从而将单次暂停的时间缩短到毫秒级，甚至微秒级。但即使是微秒级的暂停，如果每帧都发生，累积起来也会影响帧率的稳定性。

对于每秒需要渲染 30 帧或 60 帧的游戏来说，哪怕几十毫秒的暂停都可能导致明显的卡顿。比如，一个 60 FPS 的游戏，每帧的时间预算只有 16.6 毫秒。如果 GC 暂停了 30 毫秒，那么这一帧就直接“跳过”了，玩家就会感到卡顿。

### 1.4.2 GC Alloc (内存分配开销)

GC 问题的另一个重要来源是 **GC Alloc**，也就是“垃圾回收器分配内存”。这听起来有点反直觉：内存分配本身难道不是程序正常运行的一部分吗？是的，但是：

- **内存分配的开销：**在堆上分配内存本身并不是零成本的。运行时需要找到合适的空闲内存块，并进行相应的簿记（Bookkeeping）操作。虽然现代操作系统的内存分配效率很高，但频繁的分配仍然会累积成可观的开销。
- **触发 GC 的原因：**当程序不断地在堆上创建新对象时，堆内存的使用量会逐渐增加。当堆内存达到 GC 设定的阈值时，或者可用的空闲内存不足以分配新的对象时，GC 就会被触发。因此，**频繁的 GC Alloc 直接导致 GC 频繁地被触发**，进而导致更多的 GC 暂停。
- **内存膨胀：**即便对象很快被回收，如果每帧都大量分配，也会导致堆内存的峰值很高，从而使得 GC 需要处理更大的内存区域，增加 GC 的工作量和暂停时间。

所以，GC Alloc 越多，GC 就越频繁，GC 暂停就越多，最终导致卡顿越严重。因此，我们在 Unity 性能优化中，经常强调的“减少 GC”，其核心往往是“**减少 GC Alloc**”。通过减少不必要的内存分配，我们能够减少 GC 触发的频率，降低其工作量，从而提升游戏的流畅性。

## 总结

在本篇中，我们探讨了 GC 的基础概念，理解了它作为自动内存管理机制的重要性。我们了解了 GC 如何通过可达性分析来识别和回收不再使用的对象，并简要介绍了标记-清除、复制和标记-整理这些核心 GC 算法。我们还认识到，在 Unity 中，Mono GC 的阻塞特性以及频繁的 GC Alloc 是导致性能问题的关键原因。

记住，**GC 的目标是自动回收内存，但其代价是可能引发性能瓶颈**。因此，作为 Unity 开发者，我们的任务是尽量减少 GC 的工作量，让它在不影响游戏流畅性的前提下完成内存清理工作。

在接下来的第二篇中，我们将更具体地探讨在 Unity 开发中哪些常见的操作会产生 GC Alloc，以及如何利用 Unity 自带的分析工具来识别它们。敬请期待！

[Unity GC 系列教程第一篇：GC 基础概念与工作原理](#)

[Unity GC 系列教程第二篇：Unity 中常见的 GC Alloc 场景与分析工具](#)

[Unity GC 系列教程第三篇：GC Alloc 优化技巧与实践（上）](#)

[Unity GC 系列教程第四篇：GC Alloc 优化技巧与实践（下）与 GC 调优](#)

[Unity GC 系列教程第五篇：高级 GC 内核](#)

[关于我们](#)

[招贤纳士](#)

[商务合作](#)

[寻求报道](#)



400-660-0108



[kefu@csdn.net](mailto:kefu@csdn.net)



[在线客服](#)

工作时间 8:30-22:00

公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 [经营性网站备案信息](#)

[北京互联网违法和不良信息举报中心](#) [家长监护](#) [网络110报警服务](#) [中国互联网举报中心](#) [Chrome商店下载](#) [账号管理规范](#)

[版权与免责声明](#) [版权申诉](#) [出版物许可证](#) [营业执照](#) ©1999-2025北京创新乐知网络技术有限公司